

Agenda

Find middle element of LL

Merge 2 sorted LL

Sort a LL

Find cycle in LL



Revision Quizzes (1-4)

Quiz 1: Main advantage of using a linked list over an array? It can utilize all free memory without requiring contiguous space.

Quiz 2: TC to search for a value x in LL $O(N)$ as we traverse LL to search node by node

Quiz 3: Check if LL is palindrome using $O(N)$ time and $O(1)$ space?

4 → 2 → 3 → 2 → 4

4 → 2 → 3
2 → 4

① Find middle, break LL from middle

② Reverse second half of LL

h_1
4 → 2 → 3
 h_2
4 → 2

③ Compare nodes

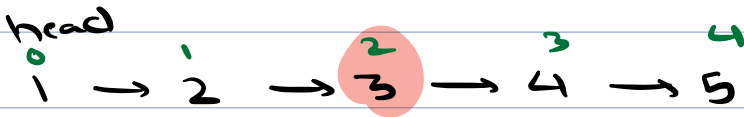
Quiz 4: TC to delete node in a LL

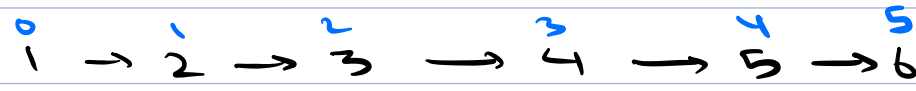
$O(N)$ as we need to traverse to prev of the target node

Quiz 1: Can we do binary search in sorted LL? No

Binary search relies on random access (especially mid) of elements.
Not supported by LL.

1. Given a linked list, find the middle element.

Ex 1 :  $N=5$ $idx=2$
ans $\rightarrow$ 3

Ex 2 :  $N=6$ $idx=2$
ans $\rightarrow$ 3 (1st middle)

Approach 1 : Get size of LL (traverse & count)

Traverse and reach mid idx

odd $\swarrow$ $N/2$
Even $\searrow$ $N/2 - 1$

Node findMiddle (Node head) {

Node temp = head

int size = 0

while (temp != NULL) {

size++

temp = temp->next

}

int midIdx = size / 2

if (size % 2 == 0)

midIdx--



temp = head

for (j=1 ; j ≤ mid $\frac{N}{2}$; j++)

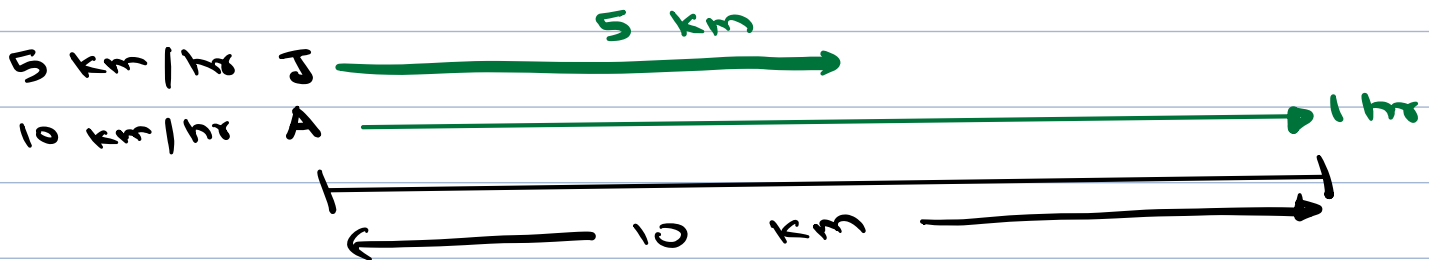
temp = temp.next

return temp

TC : $O(N + \frac{N}{2}) = O(N)$
2 passes

SC : $O(1)$

Optimized solution : 1 pass of LL



h

1 → 2 → 3 → 4 → 5 → NULL
s f

s → 1 step
f → 2 steps

For odd
f → last node

h

1 → 2 → 3 → 4 → 5 → 6 → NULL
s f

For even
f → 2nd last node

Node findMiddle (Node head) { → N/2 iterations

if (head == NULL) return NULL

Node slow = head, fast = head

while (f.next != NULL odd && f.next.next != NULL even)

slow = slow.next

fast = fast.next.next

TC: O(N)

SC: O(1)

return slow

if (a && b)

STOP if f reaches Even 2nd last node or f reaches Odd last node

Continue if f != 2nd last node AND f != Last node

2. Given two sorted linked list, merge them into a single sorted linked list.

Ex 1: $\begin{array}{c} h1 \\ 1 \rightarrow 2 \rightarrow 8 \rightarrow 10 \end{array} \quad | \quad \begin{array}{c} h2 \\ 3 \rightarrow 5 \rightarrow 9 \rightarrow 11 \end{array}$

ans: $\begin{array}{c} newH \\ 1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 11 \rightarrow NULL \end{array}$

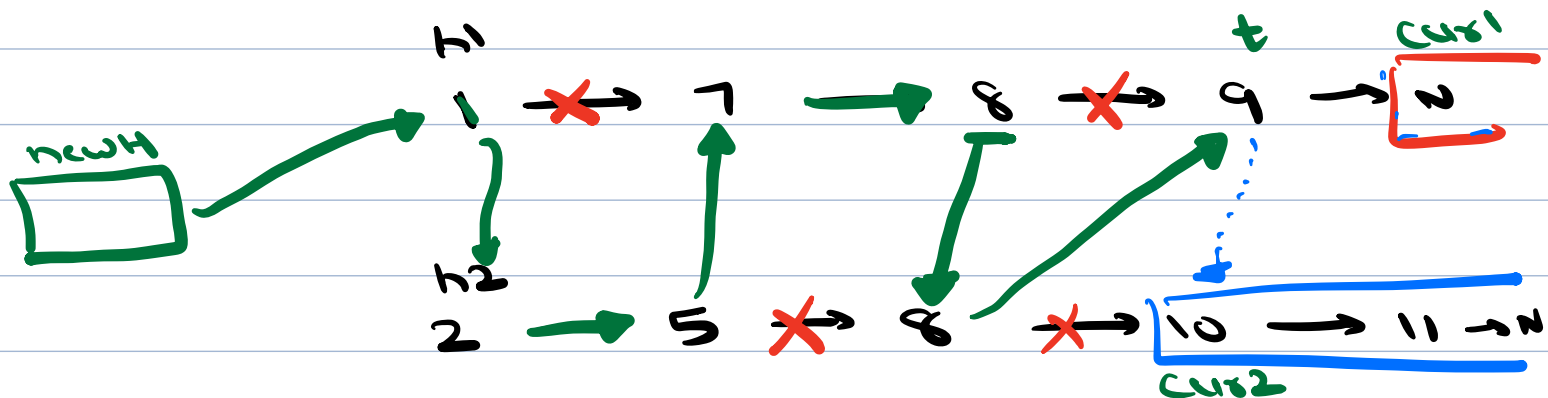
Ex 2: $\begin{array}{c} h1 \\ 1 \rightarrow 7 \rightarrow 8 \rightarrow 9 \end{array} \quad | \quad \begin{array}{c} h2 \\ 2 \rightarrow 5 \rightarrow 8 \rightarrow 10 \rightarrow 11 \end{array}$

ans: $\begin{array}{c} newH \\ 1 \rightarrow 2 \rightarrow 5 \rightarrow 7 \rightarrow 8 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 11 \rightarrow NULL \end{array}$

Quiz 5 $\begin{array}{c} h1 \\ 2 \rightarrow 10 \rightarrow 11 \end{array} \quad p1 \quad | \quad \begin{array}{c} h2 \\ 1 \rightarrow 5 \rightarrow 12 \rightarrow 15 \end{array} \quad p2$

ans: $1 \rightarrow 2 \rightarrow 5 \rightarrow 10 \rightarrow 11 \rightarrow 12 \rightarrow 15 \rightarrow NULL$

Approach: Take 2 pointers
compare nodes and keep adding smaller
nodes in final list.



```

if ( cur1.data < cur2.data ) <
    |
    | t.next = cur1
    | cur1 = cur1.next
    | t = t.next
    >

```

Final LL $\rightarrow$ newH $\rightarrow$ t

LL1 $\rightarrow$ cur1 $\rightarrow$ NULL

LL2 $\rightarrow$ cur2 $\rightarrow$ NULL

Node mergeSortedLL (Node h1, Node h2) <

Node cur1 = h1, cur2 = h2

Node newH = new Node (-1)

Node t = newH

while (cur1 != NULL && cur2 != NULL) <

if (cur1.data < cur2.data) <

t.next = cur1

cur1 = cur1.next

t = t.next

else < // cur2.data $\leq$ cur1.data

t.next = cur2

cur2 = cur2.next

t = t.next

>

if (cur1 != NULL) <

// LL2 empty

t.next = cur1

if (cur2 != NULL) <

// LL1 empty

t.next = cur2

size of LL1

TC: $O(N+M)$

return newH.next

SC: $O(1)$

size of LL2

Code works for following edge cases

① Both LL empty

② 1 LL is empty and other is non-empty

10:40

3. Sort the LL using merge sort.

Ex: $1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 3$

ans: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$

$1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 3$ $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$

$1 \rightarrow 2 \rightarrow 5$ $1 \rightarrow 2 \rightarrow 5$

$4 \rightarrow 3$ $3 \rightarrow 4$

$1 \rightarrow 2$ $1 \rightarrow 2$

5

4 3

1
2

Approach : Apply merge sort algo

Base case : Single element , return

- ① Find Mid → Divide LL into 2 pieces
- ② Individually sort both pieces
↓
Recursively call merge sort fn
- ③ Merge both sorted LL
- ④ Return

// Given head of LL, fn will sort LL and return of updated head

Node mergeSort (Node head) {

if (head == NULL || head.next == NULL) return head

Node m = findMiddle (head)

Node head2 = m.next

m.next = NULL

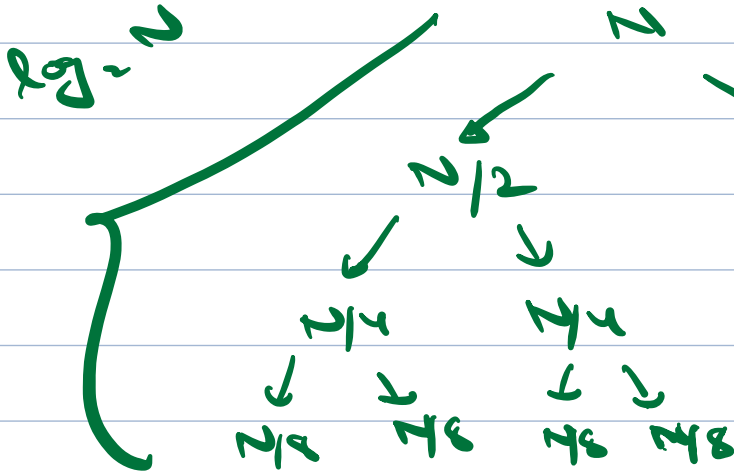
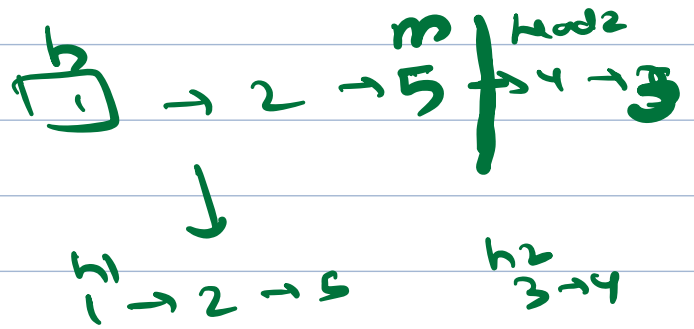
Node h1 = mergeSort (head)

Node h2 = mergeSort (head2)

return merge (h1, h2)

Pre-written
fn

Recursion



Middle
N

N

N

TC: $O(N \log_2 N)$

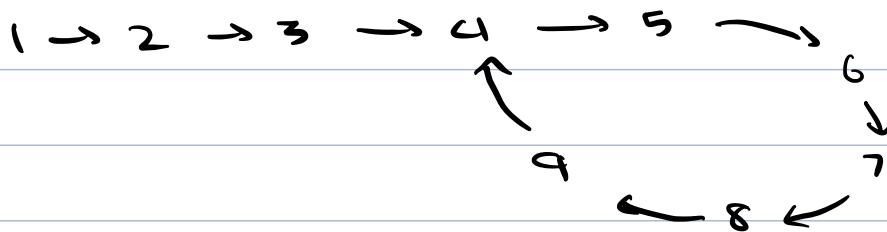
SC: $O(\log_2 N)$

Merge
N

N

N

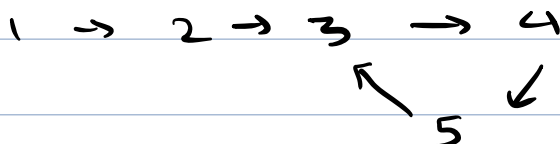
4. Detect cycle in linked list.



ans: true

1 -> 2 -> 3 -> 4 -> 5

ans: false

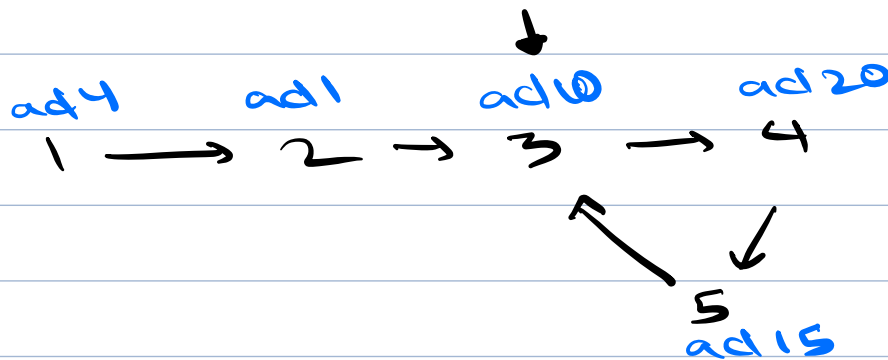


ans: true

Cycle -> some nodes are repeating

Approach 1: As we traverse LL, we maintain visited nodes in HS. If any node is already in HS, there is a cycle. Else if you reach NULL, no cycle.

HS → Node References



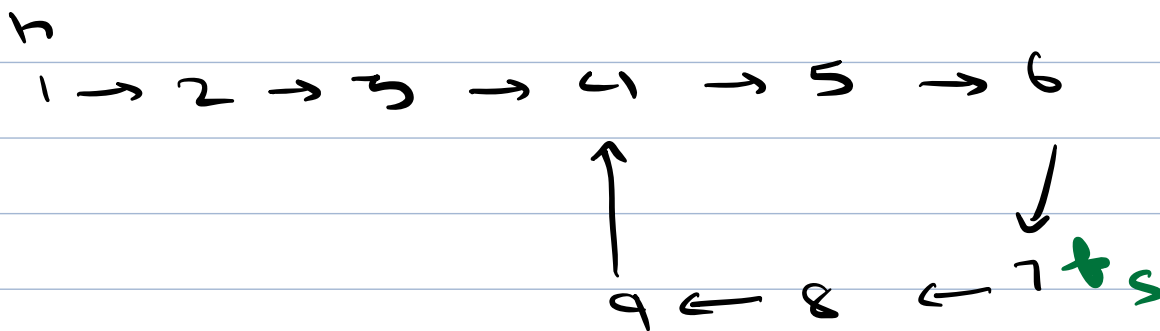
HS < Node >



TC: $O(N)$

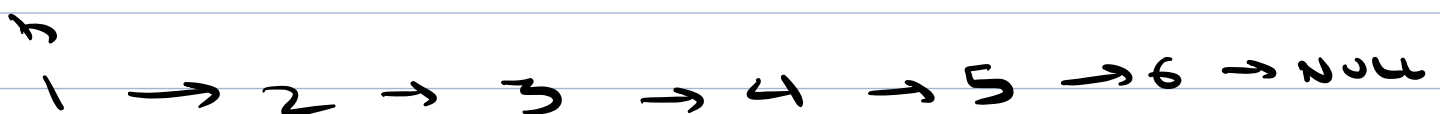
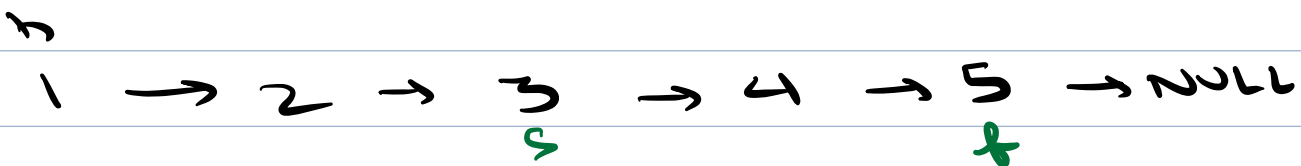
SC: $O(N)$

hs.add
(temp)



s	f	dist
4	7	3
5	9	2
6	5	1
7	1	0

Keep slow and fast at head. We move slow by 1 and fast by 2 → if they meet at any point → loop



bool hasCycle (Node head) {

Node slow = head, fast = head

while (fast != NULL && fast.next != NULL) {

slow = slow.next

fast = fast.next.next

if (slow == fast)

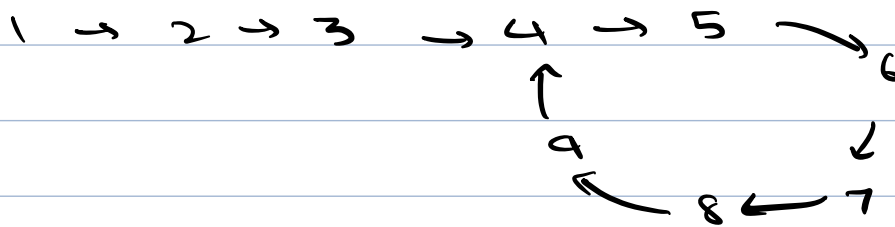
return true

return false

TC: O(N)

SC: O(1)

5. Find the ^{start} ~~meeting~~ point of the cycle if there is any cycle.

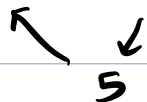


ans: 4

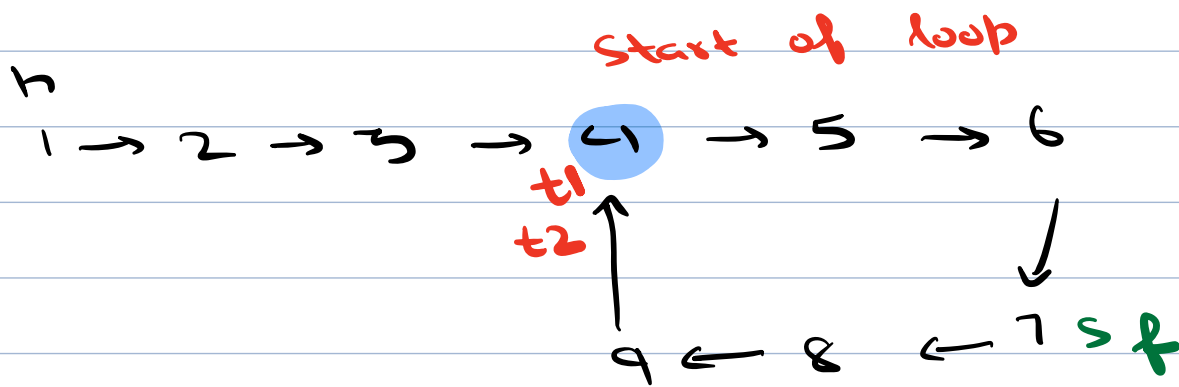
1 -> 2 -> 3 -> 4 -> 5

ans: NULL

1 -> 2 -> 3 -> 4



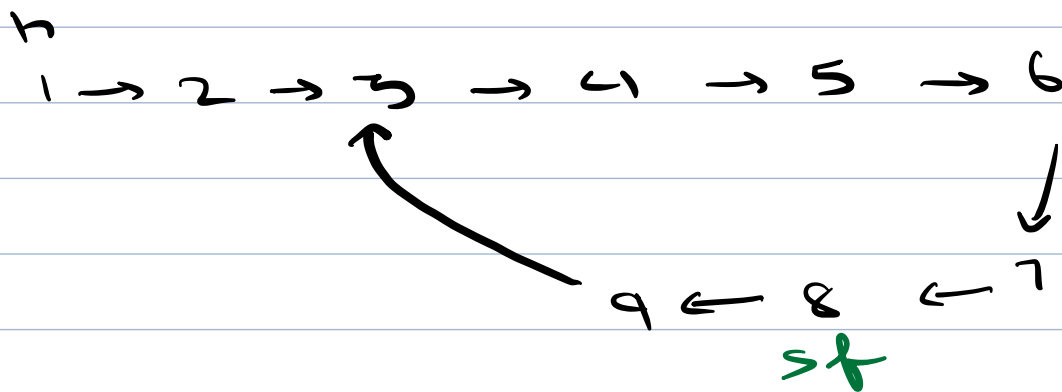
ans: 3



① Find the pt. where s and f meet
 ↓
 whether a cycle exists

② $t1 = \text{head}$, $t2 = \text{meeting point}$
 keep moving both $t1$ and $t2$ till they meet

③ $t1$ and $t2$'s point $\rightarrow$ start of cycle



Node startPointCycle(Node head) <

Node slow = head, fast = head

while (fast != NULL && fast.next != NULL) <

slow = slow.next

fast = fast.next.next

if (slow == fast) <

Node t1 = head, t2 = slow

while (t1 != t2) <

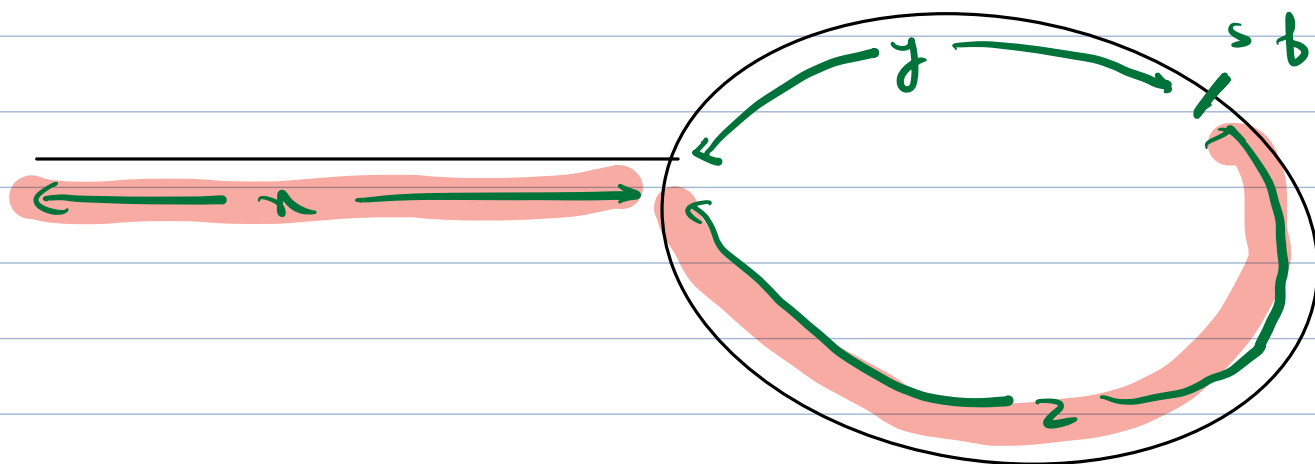
t1 = t1.next t2 = t2.next

return t1

return NULL

TC: O(N)

SC: O(1)



Dist travelled by s = n + y

Dist travelled by f = n + y + y + z

$$= x + 2y + z$$

$$\text{Dist}_f = 2 \times \text{Dist}_s$$

$$x + 2y + z = 2(x + y)$$

$$x + \cancel{2y} + z = 2x + \cancel{2y}$$

$$z = 2x - x$$

$$\Rightarrow z = x$$

HW : Break the loop